

Debian Trixie (13): installation guide with BTRFS, LUKS encryption and auto snapshots with Timeshift

Please feel free to raise any comments or issues on the [website's Github repository](#). Pull requests are very much appreciated.



Overview

In this guide, I will show you how to install Debian Trixie (13.2) with the following structure:

- An unencrypted EFI partition for the GRUB bootloader.
- A LUKS-encrypted LVM partition containing the BTRFS root filesystem.
 - The BTRFS logical volume contains a subvolume `@` for `/` and a subvolume `@home` for `/home`.
- An encrypted swap partition.
- Automatic system snapshots and easy rollback using:
 - [Timeshift](#) which will regularly take (almost instant) snapshots of the system.

This setup works similarly well on other distributions, for which I also have [installation guides](#) (with optional RAID1).

Step 0: General remarks

This tutorial uses Debian Trixie (13.2) from the [official Debian website](#), installed via a USB flash device. Other versions of Debian might also work but may require additional steps (see my other [installation guides](#)).

 I strongly advise trying the following installation steps in a virtual machine first before performing them on real hardware! For instance, you can spin up a virtual machine using the awesome [quickemu](#) project.

Step 1: Install Debian with the default installer using BTRFS

Boot from the Debian installation media and run the installer (graphical or text-based). Proceed through the standard steps:

1. Select language, location, and keyboard layout.
2. Configure network, hostname and domain name.
3. Set up users and passwords including a root password.
4. When you reach **Partition disks**, select **Guided - use entire disk and set up encrypted LVM**.
5. Select your target disk.
6. Choose the partitioning scheme: **All files in one partition**.
7. Since we chose encryption, the installer then overwrites the disk "with random data to prevent meta-inform". This might take a few minutes.
8. Enter your encryption passphrase and adjust the **Amount of volume group to use for guided partitioning** to e.g. **max**.
9. **Important:** Before continuing with the the partitioning process, double click the root partition (**LVM VG debian-vg**, **LV root**, look for `/` in the right column); then double click **Use as** and change the filesystem from **Ext4 journaling file system** to **btrfs journaling file system**.

10. Double click **Mount options** and select **relatime**, **compress**, **ssd**, and **discard** (actually these should be the default options anyways, except for **compress**)

i

- There is some debate about using **noatime** instead of **relatime**. I haven't seen a significant difference, so I use the default **relatime**.
- Since Linux kernel 6.2, BTRFS enables **discard=async** by default on devices that support TRIM operations.
- The **compress** option will be set to **zlib:3** by the installer. We will later change it to **zstd:1** for SSDs or **zstd:3** for HDDs.

11. Select **Done setting up the partition** and then **Finish partitioning and write changes to disk**. Write the changes to disk and wait for the installation to finish.

12. Finish the installation (package selection, GRUB installation, etc.).

i

The Debian installer automatically creates a BTRFS subvolume called **@rootfs** when you select BTRFS as the filesystem for **/**. After installation, we will (1) use **@** for **/** instead, and (2) add a separate **@home** subvolume for **/**. This **@** and **@home** naming convention (sometimes called *Ubuntu-style*) is required by tools like Timeshift. Other distributions use different conventions (e.g., Fedora uses **root** and **home** without the **@** prefix).

Having separate subvolumes for **/** and **/home** enables easier system rollback—you can restore system files without affecting user data. Additionally, keeping both subvolumes on a single BTRFS partition avoids the common problem of running out of space on a fixed-size root partition while the home partition has plenty of free space. These features are some of the reasons Fedora adopted BTRFS as the default filesystem.

When the installation finishes, reboot into your new system.

Step 2: Prepare BTRFS subvolumes

After booting into your new Debian system, open a terminal and become root:

```
su -
```

Examine the current setup [↗](#)

First, let's see the current partition layout:

```
lsblk
# NAME                MAJ:MIN RM   SIZE RO TYPE  MOUNTPOINTS
# sr0                  11:0    1    3.8G  0 rom
# sr1                  11:1    1  1024M  0 rom
# vda                  254:0    0    16G   0 disk
# └─vda1                254:1    0    869M  0 part  /boot/efi
# └─vda2                254:2    0    870M  0 part  /boot
# └─vda3                254:3    0   14.3G  0 part
#   └─vda3_crypt        252:0    0   14.3G  0 crypt
#     └─debian--vg-root 252:1    0   13.5G  0 lvm   /
#       └─debian--vg-swap_1 252:2    0    804M  0 lvm   [SWAP]
```

Check the current BTRFS subvolumes:

```
btrfs subvolume list /
# ID 256 gen 63 top level 5 path @rootfs
```

The installer created a subvolume called **@rootfs**. We'll create **@** as a snapshot of this and add **@home** for **/home**.

Mount the top-level BTRFS root

Mount the top-level BTRFS volume (subvolid=5) to **/mnt**:

```
mount -o subvolid=5 /dev/mapper/debian--vg-root /mnt
ls /mnt
# @rootfs
```

i **debian-vg-root:** Sometimes (e.g. in VMs) you might see the volume group name as **debian-13-vg-root** instead of **debian-vg-root**. Please adjust the commands below accordingly!

Create the @ subvolume

Create a snapshot of @rootfs called @:

```
btrfs subvolume snapshot /mnt/@rootfs /mnt/@
# Create a snapshot of '/mnt/@rootfs' in '/mnt/@'
```

Create the @home subvolume

Create the @home subvolume and move the home directory contents from the @ subvolume (**not** the @rootfs):

```
btrfs subvolume create /mnt/@home
# Create subvolume '/mnt/@home'
mv /mnt/@/home/* /mnt/@home/
```

Verify the subvolumes:

```
ls -la /mnt/@/home
# . . .
ls -la /mnt/@home
# . . . wmutschl
btrfs subvolume list /mnt
# ID 256 gen 68 top level 5 path @rootfs
# ID 257 gen 68 top level 5 path @
# ID 258 gen 68 top level 5 path @home
```

Step 3: Update fstab

Edit the fstab in the new @ subvolume to mount the correct subvolumes:

```
nano /mnt/@/etc/fstab
```

Find the line that mounts / and change it:

Before:

```
/dev/mapper/debian--vg-root /          btrfs    relatime,compress,ssd,discard,subvol=@rootfs 0      0
```

After:

```
/dev/mapper/debian--vg-root /          btrfs    relatime,compress,ssd,discard,subvol=@ 0      0
```

Add a new line for /home:

```
/dev/mapper/debian--vg-root /home      btrfs    relatime,compress,ssd,discard,subvol=@home 0      0
```

Your fstab should look similar to this:

```
cat /mnt/@/etc/fstab
# /dev/mapper/debian--vg-root /          btrfs    relatime,compress,ssd,discard,subvol=@ 0      0
# /dev/mapper/debian--vg-root /home      btrfs    relatime,compress,ssd,discard,subvol=@home 0      0
# # /boot was on /dev/vda2 during installation
# UUID=2647b326-dd45-4f11-8823-2724d7a02cb8 /boot          ext4      defaults    0      2
# # /boot/efi was on /dev/vda1 during installation
# UUID=A03F-21A1 /boot/efi      vfat      umask=0077  0      1
# /dev/mapper/debian--vg-swap_1 none          swap      sw          0      0
# /dev/sr0      /media/cdrom0  udf,iso9660 user,noauto 0      0
# /dev/sr1      /media/cdrom1  udf,iso9660 user,noauto 0      0
```

i **Compression:** Optionally, change the compress option to zstd:1 for SSDs or zstd:3 for HDDs, it is set to zlib:3 if not otherwise specified.

```
sed -i 's/compress/compress=zstd:1/' /mnt/@/etc/fstab
```

Step 4: Chroot and update initramfs and GRUB

Unmount `/mnt` and remount the `@` subvolume:

```
umount /mnt
mount -o subvol=@ /dev/mapper/debian--vg-root /mnt
```

Create a chroot environment:

```
for i in /dev /dev/pts /proc /sys /run; do mount -B $i /mnt$i; done
chroot /mnt
```

Mount all filesystems and verify:

```
mount -av
# / : ignored
# /home : successfully mounted
# /boot : successfully mounted
# /boot/efi : successfully mounted
# none : ignored
# /media/cdrom0 : ignored
# /media/cdrom1 : ignored
```

Update the initramfs and GRUB to apply the changes:

```
update-initramfs -c -k all
update-grub
```

 **Note on GRUB:** There is nothing to change in `/etc/default/grub`. Debian either relies on `/etc/fstab` for the subvolume setting or uses prober code to figure out the subvolume. Since we are in the chroot, it works without any changes.

Exit the chroot:

```
exit
```

Step 5: Reboot and verify

Reboot your system:

```
reboot
```

After logging in, become root and verify the setup:

```
su -
mount -av
# / : ignored
# /home : already mounted
# /boot : already mounted
# /boot/efi : already mounted
# none : ignored
# /media/cdrom0 : ignored
# /media/cdrom1 : ignored
```

Verify the BTRFS mount options:

```
mount -v | grep btrfs
# /dev/mapper/debian--vg-root on / type btrfs (rw,relatime,compress=zstd:1,ssd,discard,space_cache=v2,subvalid=
# /dev/mapper/debian--vg-root on /home type btrfs (rw,relatime,compress=zstd:1,ssd,discard,space_cache=v2,subvo

btrfs subvolume list /
# ID 256 gen 96 top level 5 path @rootfs
# ID 257 gen 101 top level 5 path @
# ID 258 gen 101 top level 5 path @home
```

Enable `fstrim.timer` for SSD/NVMe:

```
systemctl enable fstrim.timer
```

Update the system:

```
apt update
apt upgrade
apt dist-upgrade
apt autoremove --purge
apt autoclean
```

Optionally, delete the old `@rootfs` subvolume (after confirming everything works):

```
mount -o subvolid=5 /dev/mapper/debian--vg-root /mnt
btrfs subvolume delete /mnt/@rootfs
# Delete subvolume 256 (no-commit): '/mnt/@rootfs'
umount /mnt
btrfs subvolume list /
# ID 257 gen 101 top level 5 path @
# ID 258 gen 101 top level 5 path @home
```

Step 6: Snapshot tools

With BTRFS set up with `@` and `@home` subvolumes, you can now use various snapshot and backup tools:

- **Timeshift:** A popular GUI tool for system snapshots. Note that Timeshift **requires** the `@` and `@home` subvolume naming convention we set up in this guide.
- **grub-btrfs:** Automatically adds GRUB menu entries for your snapshots, allowing you to boot directly into any snapshot.
- **Btrbk:** A powerful tool for snapshot management and backup with support for incremental send/receive to remote locations. Personally, I prefer Btrbk for its flexibility and send/receive capabilities.
- **Snapper:** Another snapshot management tool, popular in openSUSE.

We will now focus only on Timeshift as it is (in my opinion) the most user-friendly one.

Install and configure Timeshift

```
apt install -y timeshift
```

Open Timeshift from Applications menu and configure it:

- Select **BTRFS** as the **Snapshot Type**; click **Next**.
- Select your BTRFS system partition as the **Snapshot Location**; click **Next**. Ignore the warning “Selected device does not have BTRFS partition”-this is a bug with encrypted devices.
- **Select Snapshot Levels** (my recommendations):
 - Activate **Monthly** and set to 2.
 - Activate **Weekly** and set to 3.
 - Activate **Daily** and set to 5.
 - Deactivate **Hourly**.
 - Activate **Boot** and set to 5.
 - Click **Next**.
- Include the `@home` subvolume.
- Click **Finish**.

Create a manual snapshot

Simply click the “Create” button and name the label to “Clean Install” (or whatever you want).

Verify snapshots

In a terminal with root privileges, you can verify the snapshots:

```
btrfs subvolume list /
# ID 257 gen 300 top level 5 path @
# ID 258 gen 300 top level 5 path @home
# ID 259 gen 295 top level 5 path timeshift-btrfs/snapshots/2025-12-10_10-00-00/@
# ID 260 gen 295 top level 5 path timeshift-btrfs/snapshots/2025-12-10_10-00-00/@home
```

Step 7: Practice recovery and system rollback

Let’s simulate a system failure to practice recovery.

Delete the **/etc** folder and reboot (simulating disaster):

```
rm -rf /etc
reboot now
```

If you try to reboot, the system will fail.

Rollback with Timeshift from live environment

1. Boot from the Debian live installer.
2. Open **Files** and click on the encrypted partition. Enter your passphrase which will mount the partition even though there might be an error message.
3. Open a terminal and check whether the partition is unlocked:

```
sudo -i
ls /dev/mapper/
# control    debian--vg-root    debian--vg-swap_1    luks- [UUID-OF-ENCRYPTED-PARTITION]
```

4. Install Timeshift:

```
apt update
apt install -y timeshift
```

5. Open Timeshift from Applications menu and select **BTRFS**, and choose your disk. Ignore the warning “Selected device does not have BTRFS partition”-this is a bug with encrypted devices.
6. Select a snapshot and click **Restore**.
7. Reboot.

Rollback manually

1. Boot from the Debian live USB, unlock, and activate LVM in the terminal:

```
sudo -i
cryptsetup luksOpen /dev/vda3 vda3_crypt
mount -o subvolid=5 /dev/mapper/debian--vg-root /mnt
ls /mnt
# @ @home timeshift-btrfs
```

2. Move the broken subvolume:

```
mv /mnt/@ /mnt/@.broken
```

3. Find a valid snapshot:

```
ls /mnt/timeshift-btrfs/snapshots
```

4. Create a snapshot of the good subvolume as the new **@**:

```
btrfs subvolume snapshot /mnt/timeshift-btrfs/snapshots/2025-12-10_10-00-00/@ /mnt/@
```

5. Reboot.

Once back in your system, delete the broken subvolume:

```
mount -o subvolid=5 /dev/mapper/debian--vg-root /mnt
btrfs subvolume delete /mnt/@.broken
umount /mnt
```

FINISHED! CONGRATULATIONS!

PREVIOUS

Debian Trixie (13): Things to do after installation (Apps, Settings, and Tweaks)

NEXT

Last updated on Dec 10, 2025

[Privacy Policy](#)

© 2026 Willi Mutschler. This work is licensed under [CC BY SA 4.0](#)



Published with [Wowchemy](#) — the free, [open source](#) website builder that empowers creators.